

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

import statsmodels.api as sm

from scipy import stats

%matplotlib inline
```

```
df = pd.read_csv("marketing_and_sales_data_evaluate_lr.csv")

df.head()
```

	TV	Radio	Social_Media	Sales
0	16.0	6.566231	2.907983	54.732757
1	13.0	9.237765	2.409567	46.677897
2	41.0	15.886446	2.913410	150.177829
3	83.0	30.020028	6.922304	298.246340
4	15.0	8.437408	1.405998	56.594181

```
print(df.shape)
```

```
df.info()
```

```
(4572, 4)
<class 'pandas.DataFrame'>
RangeIndex: 4572 entries, 0 to 4571
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  -
0    TV              4562 non-null   float64
1    Radio           4568 non-null   float64
2    Social_Media    4566 non-null   float64
3    Sales           4566 non-null   float64
dtypes: float64(4)
memory usage: 143.0 KB
```

```
df.isnull().sum()
```

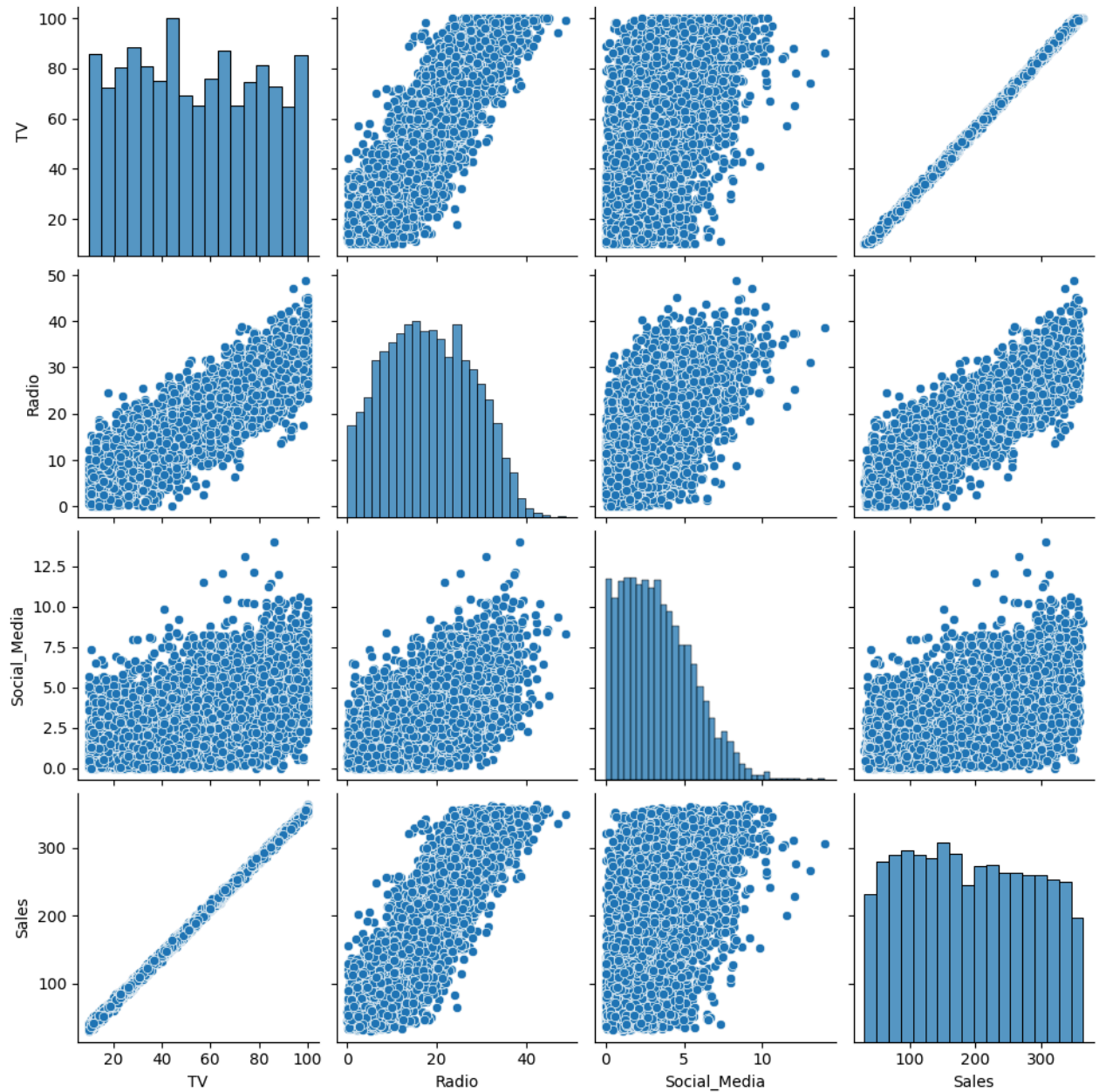
```
TV              10
Radio            4
Social_Media     6
Sales            6
dtype: int64
```

```
df.describe()
```

	TV	Radio	Social_Media	Sales
count	4562.000000	4568.000000	4566.000000	4566.000000
mean	54.066857	18.160356	3.323956	192.466602
std	26.125054	9.676958	2.212670	93.133092
min	10.000000	0.000684	0.000031	31.199409
25%	32.000000	10.525957	1.527849	112.322882
50%	53.000000	17.859513	3.055565	189.231172
75%	77.000000	25.649730	4.807558	272.507922
max	100.000000	48.871161	13.981662	364.079751

```
sns.pairplot(df)
```

```
plt.show()
```



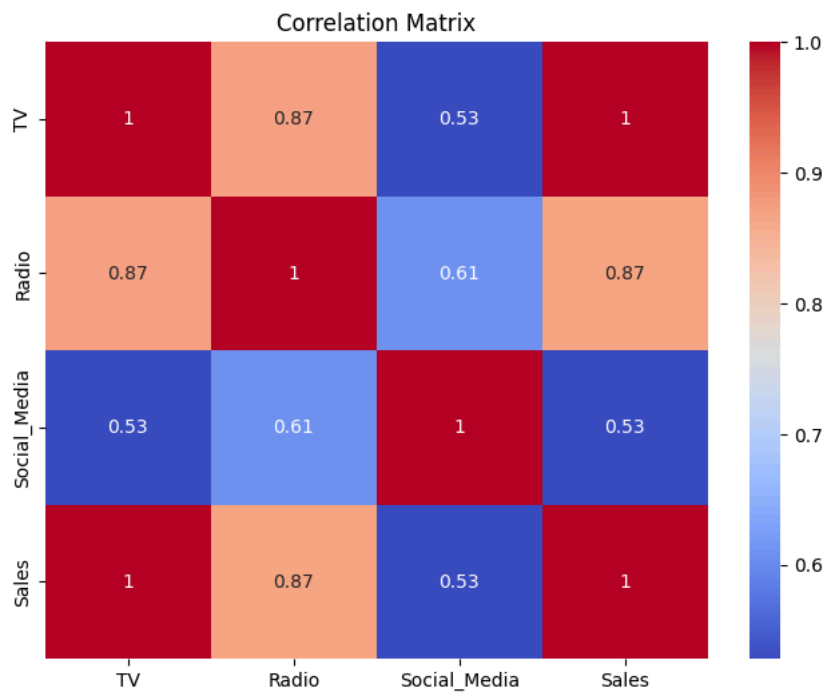
```
corr = df.corr()

plt.figure(figsize=(8,6))

sns.heatmap(
    corr,
    annot=True,
    cmap="coolwarm"
)

plt.title("Correlation Matrix")

plt.show()
```



```
corr["Sales"].sort_values(ascending=False)
```

```
Sales      1.000000
TV          0.999497
Radio       0.869105
Social_Media 0.528906
Name: Sales, dtype: float64
```

```
X = pd.to_numeric(df["TV"], errors="coerce")
```

```
y = pd.to_numeric(df["Sales"], errors="coerce")
```

```
print(X.isnull().sum())
print(y.isnull().sum())
```

```
10
6
```

```
X = X.dropna()
y = y.loc[X.index]
```

```
import numpy as np

print(np.isinf(X).sum())
print(np.isinf(y).sum())
```

```
0
0
```

```
X = sm.add_constant(X)
```

```
print(X.isnull().sum())

print(np.isinf(X).sum())

print(X.head())
```

```
const      0
TV          0
dtype: int64
const      0
TV          0
dtype: int64
const      TV
0      1.0  16.0
1      1.0  13.0
2      1.0  41.0
3      1.0  83.0
4      1.0  15.0
```

```

model = sm.OLS(y, X)

results = model.fit()

print(results.summary())

```

```

=====
                        OLS Regression Results
=====
Dep. Variable:          Sales      R-squared:                nan
Model:                  OLS        Adj. R-squared:            nan
Method:                 Least Squares    F-statistic:              nan
Date:                  Sat, 13 Jun 2026    Prob (F-statistic):        nan
Time:                  00:05:28      Log-Likelihood:            nan
No. Observations:      4562         AIC:                      nan
Df Residuals:          4560         BIC:                      nan
Df Model:              1
Covariance Type:       nonrobust
=====
                        coef    std err          t      P>|t|      [0.025    0.975]
-----
const                nan         nan         nan         nan         nan         nan
TV                   nan         nan         nan         nan         nan         nan
=====
Omnibus:              nan    Durbin-Watson:            nan
Prob(Omnibus):        nan    Jarque-Bera (JB):          nan
Skew:                 nan    Prob(JB):                  nan
Kurtosis:             nan    Cond. No.                  138.
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```

import numpy as np
import statsmodels.api as sm

X = pd.to_numeric(df["TV"], errors="coerce")
y = pd.to_numeric(df["Sales"], errors="coerce")

data = pd.DataFrame({
    "TV": X,
    "Sales": y
})

data = data.replace([np.inf, -np.inf], np.nan)

data = data.dropna()

X = data["TV"]
y = data["Sales"]

X = sm.add_constant(X)

model = sm.OLS(y, X)

results = model.fit()

print(results.summary())

```

```

=====
                        OLS Regression Results
=====
Dep. Variable:          Sales      R-squared:                0.999
Model:                  OLS        Adj. R-squared:            0.999
Method:                 Least Squares    F-statistic:              4.527e+06
Date:                  Sat, 13 Jun 2026    Prob (F-statistic):        0.00
Time:                  00:05:29      Log-Likelihood:            -11393.
No. Observations:      4556         AIC:                      2.279e+04
Df Residuals:          4554         BIC:                      2.280e+04
Df Model:              1
Covariance Type:       nonrobust
=====
                        coef    std err          t      P>|t|      [0.025    0.975]
-----
const             -0.1263      0.101     -1.257     0.209     -0.323     0.071
TV                3.5614      0.002    2127.776     0.000     3.558     3.565
=====
Omnibus:              0.051    Durbin-Watson:            2.002
Prob(Omnibus):        0.975    Jarque-Bera (JB):          0.030
Skew:                 0.001    Prob(JB):                  0.985
Kurtosis:             3.012    Cond. No.                  138.
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```

print(results.summary())

```

OLS Regression Results

```
=====
Dep. Variable:          Sales    R-squared:                0.999
Model:                  OLS      Adj. R-squared:            0.999
Method:                 Least Squares    F-statistic:          4.527e+06
Date:                   Sat, 13 Jun 2026    Prob (F-statistic):    0.00
Time:                   00:05:30    Log-Likelihood:        -11393.
No. Observations:       4556    AIC:                   2.279e+04
Df Residuals:           4554    BIC:                   2.280e+04
Df Model:                1
Covariance Type:        nonrobust
=====
               coef      std err          t      P>|t|      [0.025      0.975]
-----
const         -0.1263      0.101        -1.257      0.209      -0.323      0.071
TV             3.5614      0.002     2127.776      0.000      3.558      3.565
=====
Omnibus:                 0.051    Durbin-Watson:           2.002
Prob(Omnibus):            0.975    Jarque-Bera (JB):          0.030
Skew:                     0.001    Prob(JB):                  0.985
Kurtosis:                 3.012    Cond. No.                  138.
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
predictions = results.predict(X)
```

```
predictions[:5]
```

```
0    56.856252
1    46.172019
2   145.891529
3   295.470794
4    53.294841
dtype: float64
```

```
plot_data = pd.DataFrame({
    "TV": data["TV"],
    "Sales": data["Sales"],
    "Predictions": predictions
})

plot_data = plot_data.sort_values("TV")

plt.figure(figsize=(8,5))

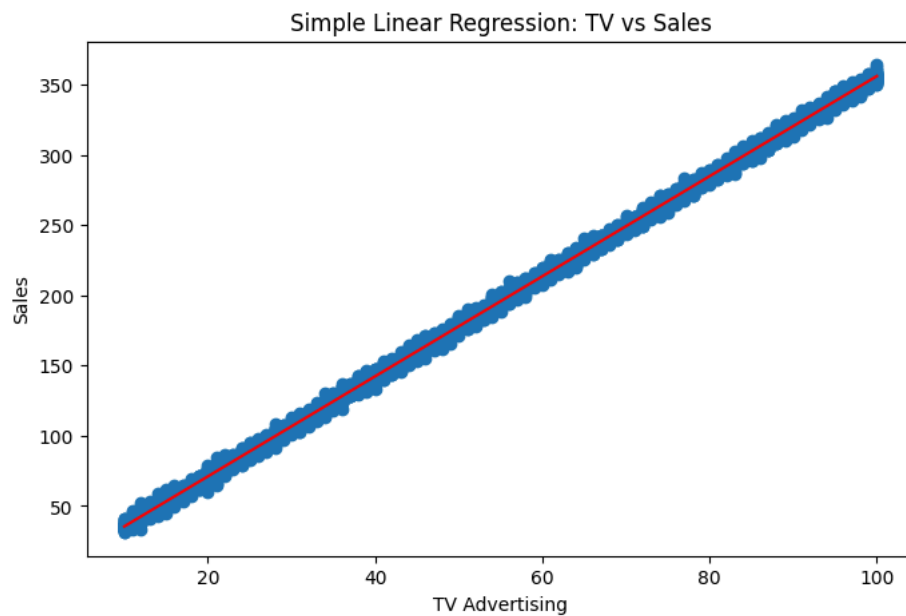
plt.scatter(
    plot_data["TV"],
    plot_data["Sales"]
)

plt.plot(
    plot_data["TV"],
    plot_data["Predictions"],
    color="red"
)

plt.xlabel("TV Advertising")
plt.ylabel("Sales")

plt.title("Simple Linear Regression: TV vs Sales")

plt.show()
```



```
plt.figure(figsize=(8,5))

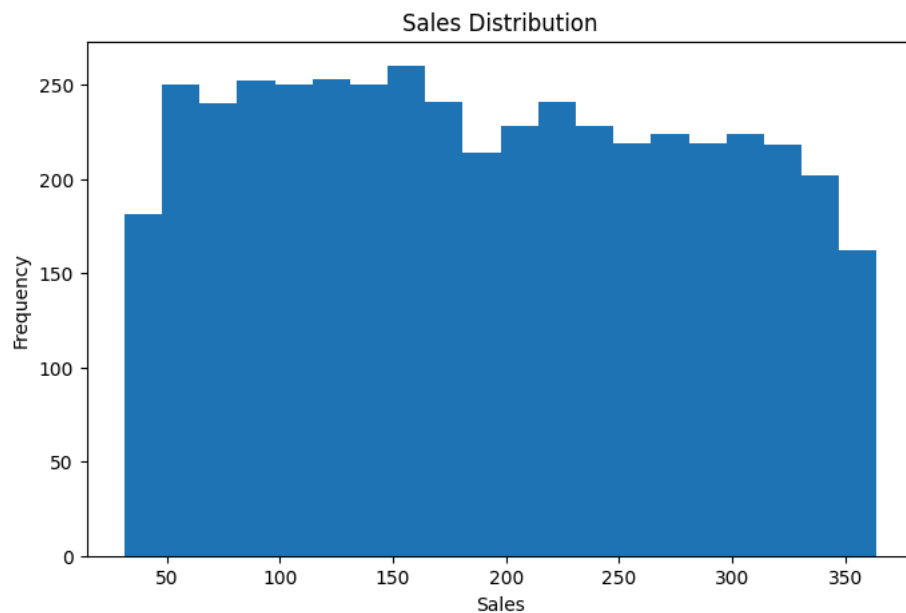
plt.hist(
    y,
    bins=20
)

plt.title("Sales Distribution")

plt.xlabel("Sales")

plt.ylabel("Frequency")

plt.show()
```



```
residuals = data["Sales"] - predictions
residuals.head()
```

```
0    -2.123495
1     0.505878
2     4.286300
3     2.775546
4     3.299340
dtype: float64
```

```
print(len(data["TV"]))
print(len(data["Sales"]))
print(len(predictions))
```

4556
4556
4556

```
plt.figure(figsize=(8,5))

plt.scatter(
    predictions,
    residuals
)

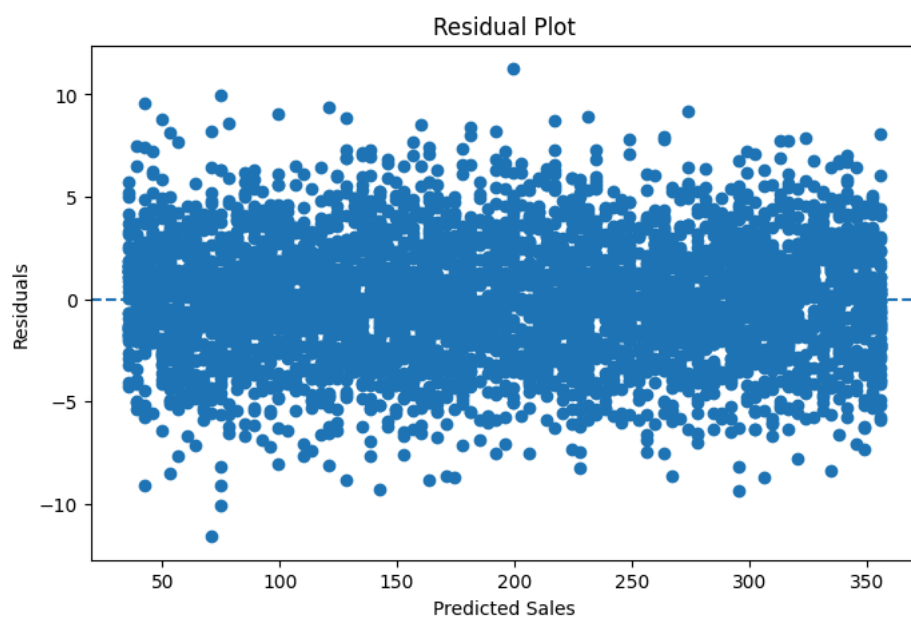
plt.axhline(
    y=0,
    linestyle="--"
)

plt.xlabel("Predicted Sales")

plt.ylabel("Residuals")

plt.title("Residual Plot")

plt.show()
```



```
plt.figure(figsize=(8,5))

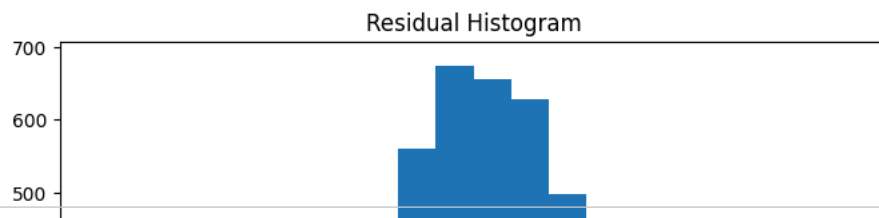
plt.hist(
    residuals,
    bins=20
)

plt.title("Residual Histogram")

plt.xlabel("Residual")

plt.ylabel("Frequency")

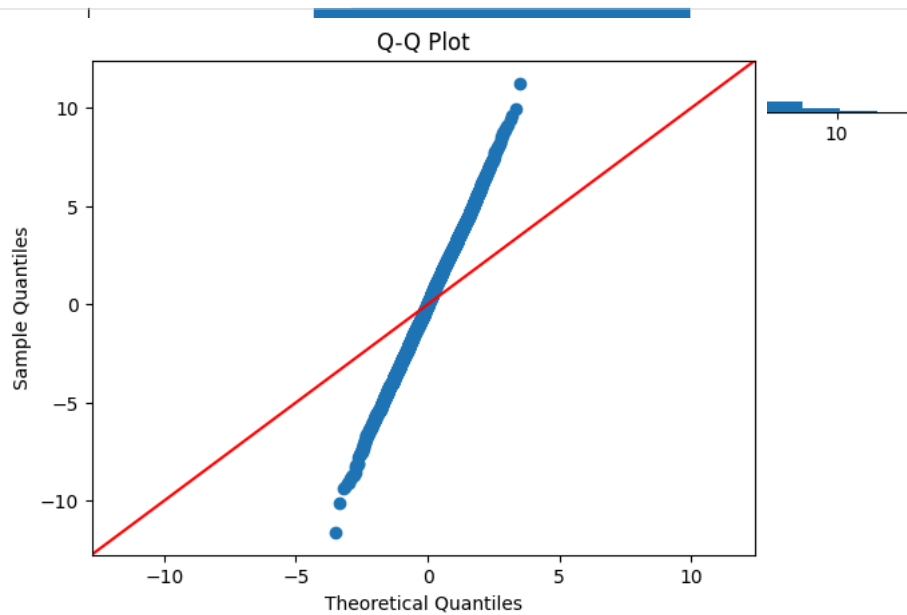
plt.show()
```



```
sm.qqplot(
    residuals,
    line="45"
)

plt.title("Q-Q Plot")

plt.show()
```



```
intercept = results.params.iloc[0]

slope = results.params.iloc[1]

print("Intercept:", intercept)

print("Slope:", slope)
```

```
Intercept: -0.1263253919483065
Slope: 3.561411078103537
```

Model Interpretation

The coefficient of TV advertising is positive.

The coefficient of TV advertising is positive. This means that for every unit increase in TV advertising, the response variable increases by approximately 3.56 units, holding all other variables constant.